

SYNOPSIS :

SOFTWARE ENGINEERING SUBJECT

Focuses on data engineering pipelines, system design, UML specifications, design patterns, and robust storage orchestration.

1. Project Title & Abstract

- **Title:** Architecture, Design Patterns, and Scalable Data Engineering Pipelines for an Enterprise Graph-Driven Rare Disease Diagnostic Engine
- **Domain:** Software Engineering, Data Engineering, ETL Pipeline Design, Graph Database Architectures
- **Objective:** Apply rigorous software engineering and data pipelining principles to build a reliable, maintainable, and deterministic ETL system that merges relational patient cohorts with massive biological knowledge bases.

2.1 Decoupled Ingestion Framework

- **Data Extraction Component:** A modular ingestion layer built using the **Strategy Design Pattern**. It isolates the distinct parsing algorithms required to ingest different functional genomic formats (Direct coding variants, non-coding miRNA vectors, Silent structural profiles, Missing/corrupted files, and Epigenetic markers).
- **Data Quality Gateways:** Integrates programmatic assertion checks via data validation frameworks (such as **Great Expectations**). This enforces structural integrity and handles null-value masking automatically before graph writing operations.

2.2 Spark Transformation & Alignment Engine

- **Distributed Processing Layer:** Employs an optimized data processing engine (such as **Apache Spark**) to load and restructure PrimeKG's 4 million biological relation records.
- **ID Mapping & Normalization Subsystem:** Implements a highly scalable transformation step using the **Façade Design Pattern**. This subsystem unifies disparate biomedical vocabularies (OMIM, Entrez, Ensembl, HPO) and maps the custom patient attributes cleanly to existing PrimeKG destination graph keys.

2.3 Storage Orchestration (Database Access Layer)

- **Dual-Persistence Target Layer:** Downstream data flows into a multi-model storage layer.
 - A relational structure (e.g., PostgreSQL) caches pipeline logs and job metadata.
 - A native Graph Database (e.g., **Neo4j** or AWS Neptune) stores the combined network topology.
- **Object-Graph Mapper (OGM) Design:** Implements a dedicated Graph Database Access Object (DAO) pattern to batch write records safely, preventing thread contention during massive edge generation tasks.

2.4 Pipeline Coordination & Orchestration

- **DAG Scheduling Engine:** Utilizes **Apache Airflow** or Prefect to model the entire workflow as an explicit Directed Acyclic Graph (DAG).
- **Task Failure Strategies:** Standardizes recovery mechanics through programmatic task retries, data checkpointing, and

backfill scripts. These features prevent full pipeline restarts if individual node mappings or graph writes fail.

3. Engineering Metrics & Quality Assurance

- **Testing Strategy:**
 - *Unit Testing:* 100% code coverage on schema-validation scripts using `pytest`.
 - *Integration Testing:* Verification of graph queries and batch writing pipelines against an isolated test database instance.
 - *Data Integrity Testing:* Verifying graph density invariants (ensuring every `Patient` node correctly instantiates target links without corrupting background PrimeKG paths). [[1](#)]
 - **Pipeline Monitoring:** Real-time logging of step durations and partition write speeds to proactively detect processing bottlenecks across the graph-construction pipeline.
-